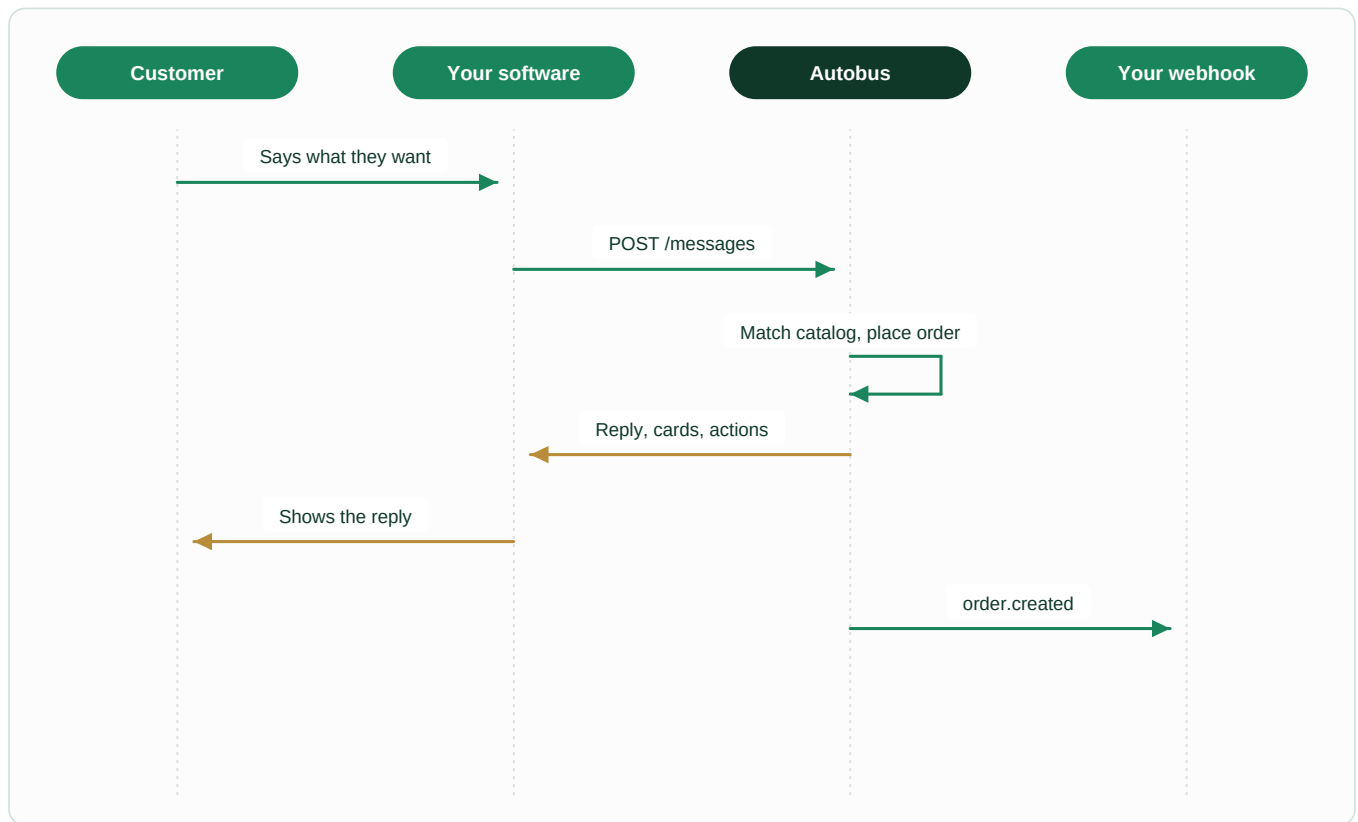




Connect your software to Autobus

The customer talks to your product. Your product asks Autobus. Autobus answers from the catalog and, when the customer confirms, creates the order.



You connect from the Autobus web portal. The mobile app does not manage this integration.

Many stores under one business

A platform such as Shopify keeps one API key. Each call names the store. Autobus answers from that store only, and sends the same store id back.

Your platform

One API key for the parent business.

You already know which store the customer is in.



Mark the store

Send `sub_business.external_id`, for example `store_12`, with the catalog and with each message.



Reply for that store

The response, the order, and the webhook all include the same `sub_business`.

Route that payload back to `store_12`. Another store never sees this catalog or this chat.

Two kinds of answers

A price or a product comes from the catalog. A question about the business comes from memory you train on the web portal.

**Catalog**

What you sell, the price, and the stock.

Enter it in the portal, or push it from your software when catalog mode is synced.

The bot will not quote a price that is not in this catalog.

Business memory

Hours, location, policies, and how you work.

Train this under AI Intelligence on the Autobus web portal.

The embed API does not accept a dump of business facts on each message.

Where you set it up**Web portal only**

Sign in at the Autobus web portal.

Open Settings, then Embedded chat.

Choose the catalog mode, paste your webhook URL, issue the API key, and turn messages on.

**Train the business memory first**

Open AI Intelligence on the same web portal.

Save the business profile and any documents you want the bot to use.

After that, customer questions about the business can be answered in your embedded chat.

What you send, in order**A. Issue the key**

Settings → Embedded chat on the web portal.

The full key is shown once. Send it as Authorization: Bearer ab_live_...

**B. Optional catalog push**

Skip this when products are managed in the portal.

When mode is synced, PUT /api/v1/embed/catalog with your SKUs.

**C. Each customer message**

POST /api/v1/embed/messages

One message in. One reply out. Reuse message.id if you have to retry.

**D. Your webhook**

Autobus POSTs order.created when an order is placed.

You later POST the order number when you have taken payment or shipped.

Catalog

PUT /api/v1/embed/catalog only works when the portal catalog mode is synced. Managed mode returns 409.

Up to 200 items per call. The same external_id overwrites the previous item.



```
{
  "items": [{
    "external_id": "SKU-RICE-5",
    "kind": "product",
    "name": "Rice Bag 5kg",
    "price": {"amount": "75.00", "currency": "GHS"},
    "stock": {"tracked": true, "quantity": 40},
    "active": true
  }]
}
```

Field	Required	Meaning
external_id	yes	Your SKU. Later upserts with this id replace the item.
kind	no	product, or service.
name	yes	The name the bot may say and sell.
price.amount	no	The price the bot quotes. It does not take a price from the chat.
stock.tracked	no	true counts stock. false is a service that can always be ordered.
stock.quantity	when tracked	On hand. Zero means the bot will not sell it.
active	no	false hides it from the bot and keeps old orders.

A customer message

```
POST /api/v1/embed/messages
Authorization: Bearer ab_live_...
```

```
{ "conversation_id": "thread_441",
  "customer": {"external_id": "cust_8841", "name": "Ama Mensah"},
  "message": {"id": "msg_91", "text": "I want 2 bags of rice" } }
```

Field	Required	Meaning
customer.external_id	yes	Stable id in your software. This keys the conversation.
customer.name / phone	no	Copied onto the order when the customer has not typed them.
message.id	yes	Retrying the same id returns the first response and does not place a second order.
message.text	yes	What the customer said.

The reply has text for the customer, product cards, and an actions list. Show the text. If actions contains order.created, record the order number and each item external_id. actions is empty when the bot only answered. conversation.handoff means the customer asked for a person. Payment stays in your software. The order is created unpaid and also appears in the portal.

```
{ "reply": { "text": "Order placed for 2 x Rice Bag 5kg...", "products": [] },
  "actions": [{ "type": "order.created",
    "order": { "order_number": "ORD-...", "source": "embed",
      "items": [{"external_id": "SKU-RICE-5", "quantity": 2}],
      "total": "150.00", "payment_status": "pending" } } ] }
```

Webhooks and order updates



Autobus calls you

POST to the webhook URL saved on the portal.

Events: order.created, order.updated, conversation.handoff, catalog.sync_failed.

Header X-Autobus-Signature is sha256= plus HMAC-SHA256 of the raw body, using the portal webhook secret.

Check the raw bytes before you parse JSON. This version does not retry, so also trust the actions array on the chat response.



You call Autobus after you fulfill

POST /api/v1/embed/orders/{order_number}

payment_status: pending, paid, partial, refunded, failed.

fulfillment_status: unfulfilled, partial, fulfilled, shipped, delivered.

Autobus then sends order.updated.

HTTP	When
401	Missing or unknown API key, or embedded chat is off.
400	A required chat or catalog field is missing or invalid.
404	That order number is not on this business.
409	Catalog push while the portal mode is still managed in Autobus.